

Models and Parameters for Developers

Generative Models for Developers

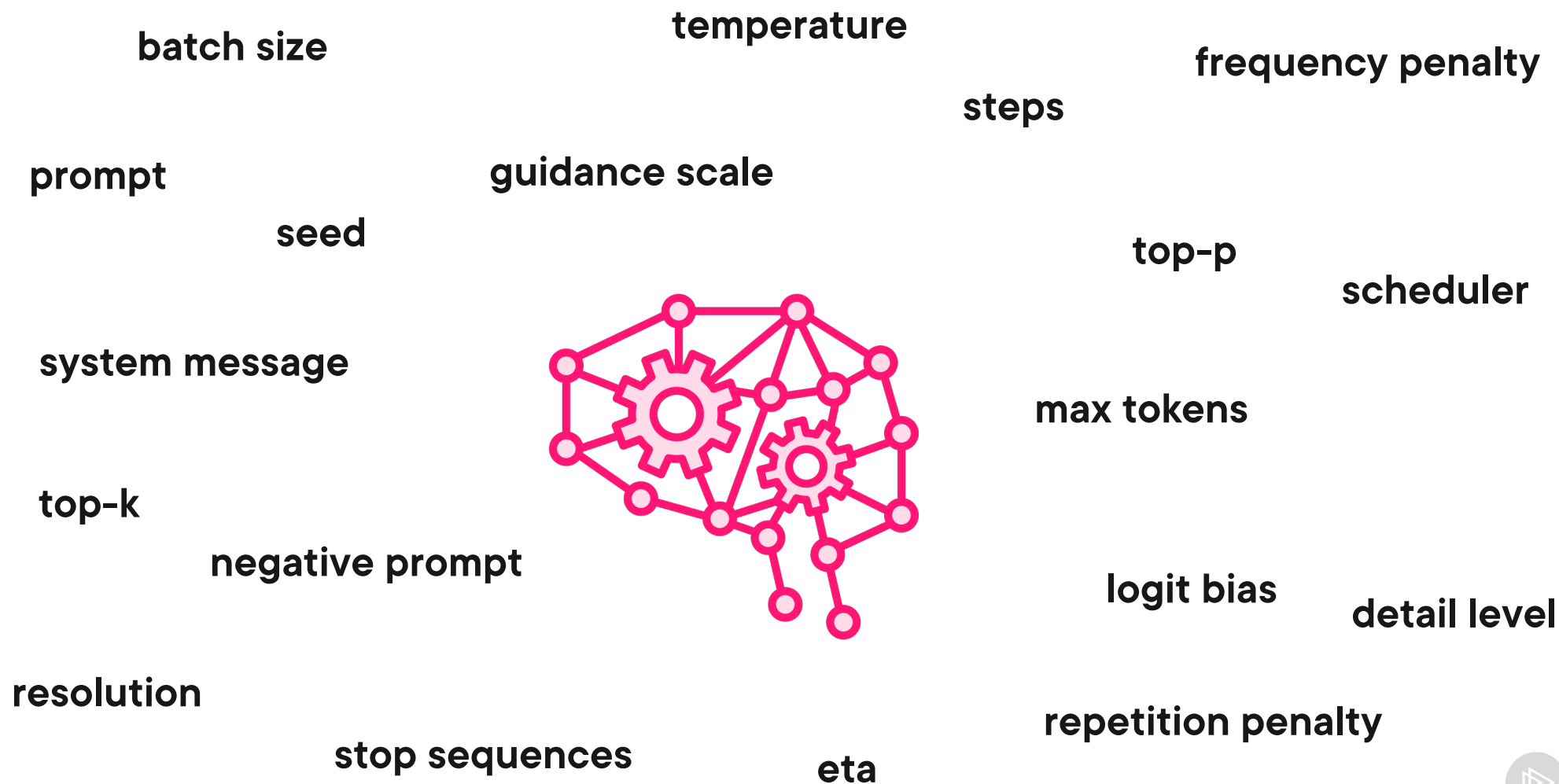


Daryle Serrant

Machine Learning Engineer

@daryleserrant | www.daryleserrant.com







Transformer Architectures



What Are Transformers?



Lorem ipsum dolor sit
amet...



Summaries
Translations
Conversations



Transformer Architecture

Transformer Layers

Embedding + Position Encoding

Tokenizer

She has many chocolates in her

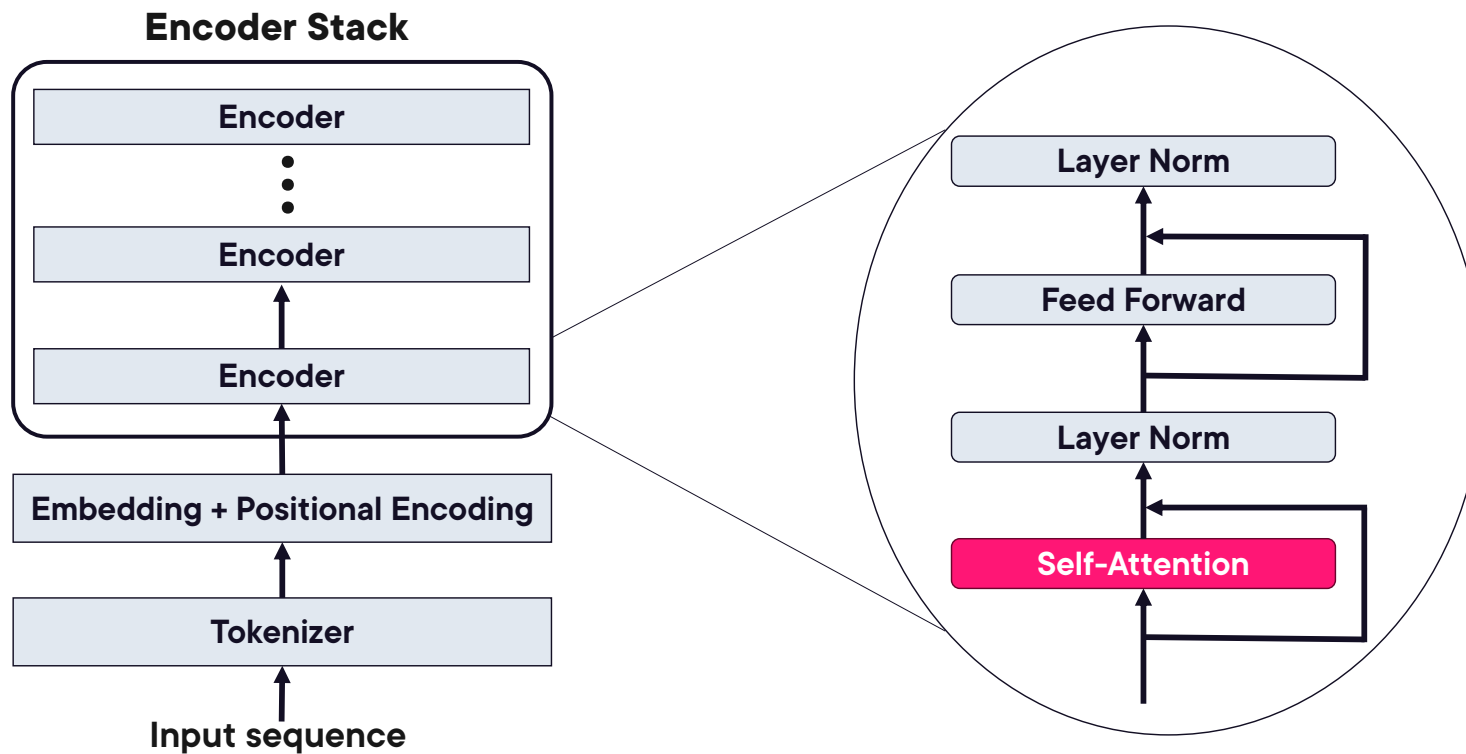
Encoders

Decoders

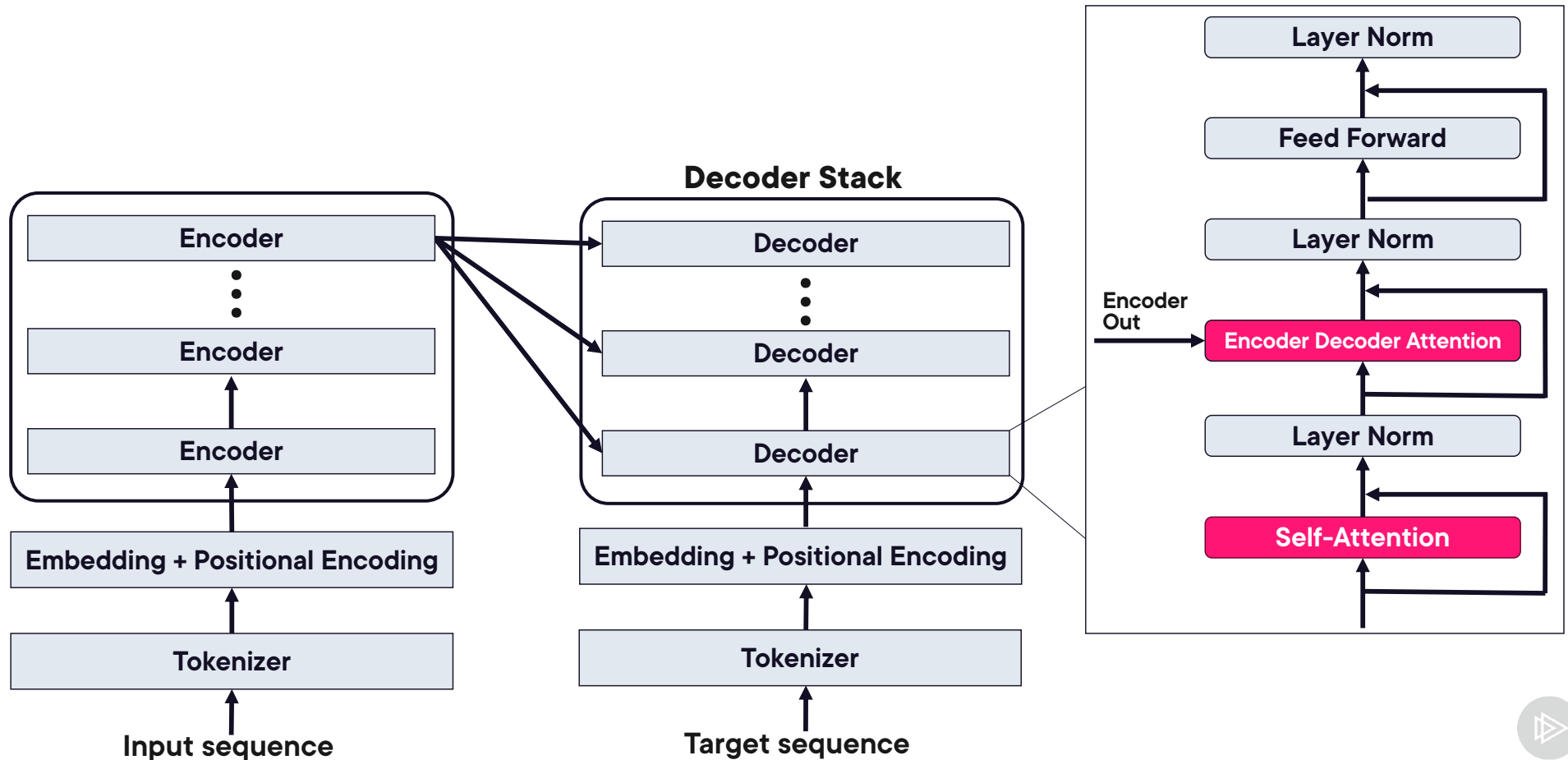
Encoders + Decoders



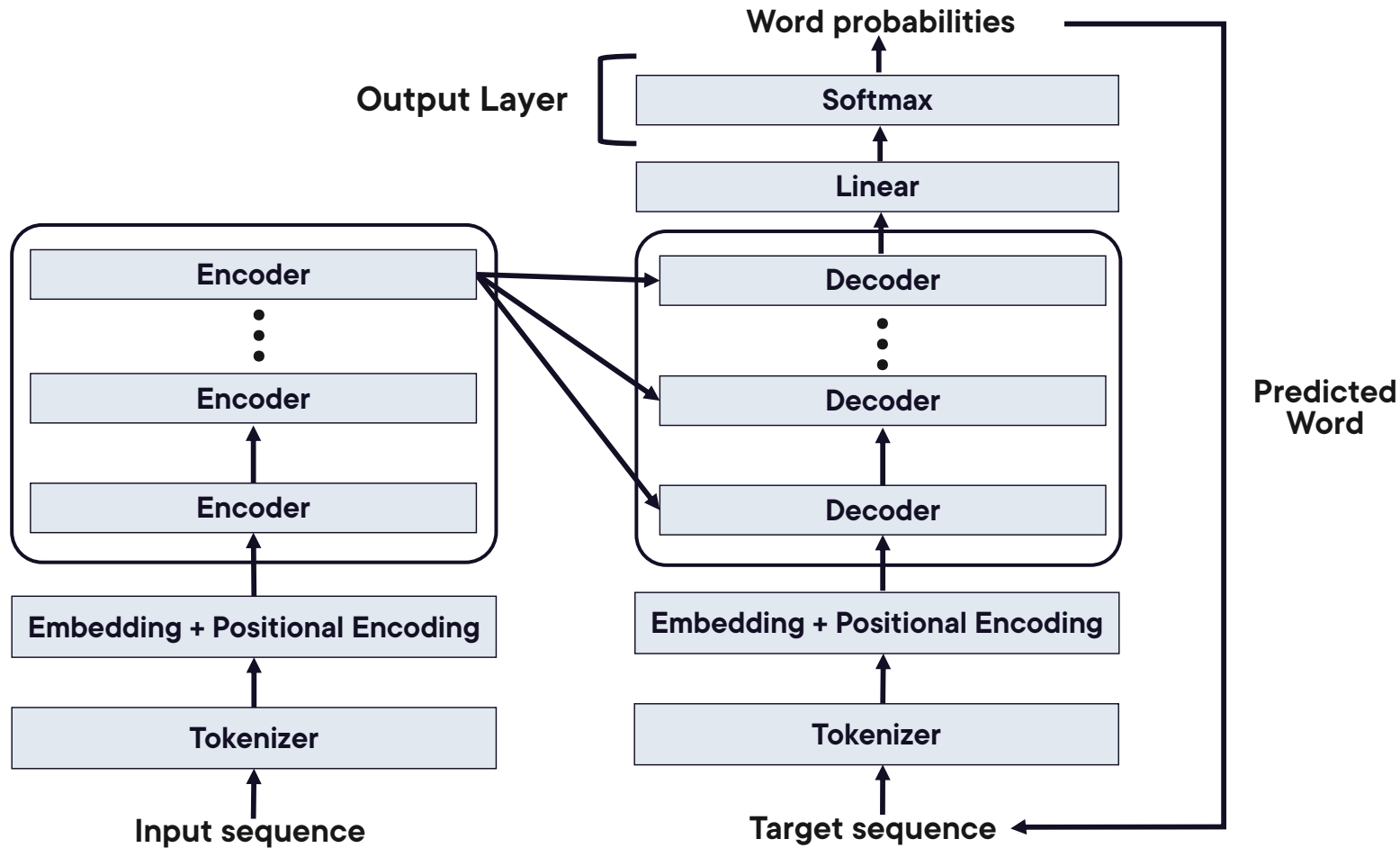
Transformer Architecture



Transformer Architecture



Transformer Architecture



Types of Transformers

**Encoder-only
architecture**

**Decoder-only
architecture**

**Encoder-decoder
architecture**





BETHANY'S

PIE SHOP

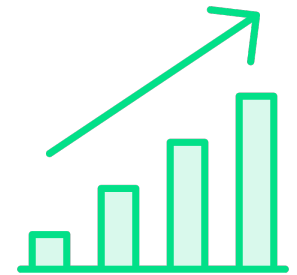




+



=



Bethany's Pie Shop

Generative AI

**Better customer
experience**

Growth



Encoder-only Transformer

Analyze customer reviews

Categorize feedback

Bidirectional Encoder Representations from
Transformers (BERT)

Produce embedding representation of input



Encoder-only Transformer

Analyze customer reviews

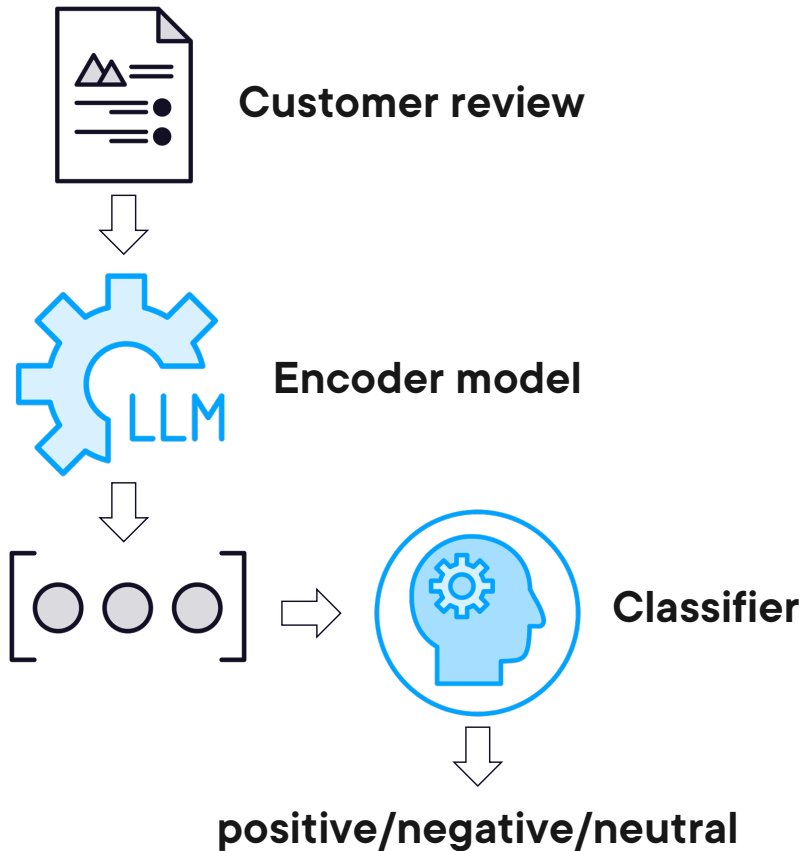
Categorize feedback

Bidirectional Encoder Representations from
Transformers (BERT)

Produce embedding representation of input



Encoder-only Transformer



Analyze customer reviews

Categorize feedback

Bidirectional Encoder Representations from Transformers (BERT)

Produce embedding representation of input

Other use cases

- Information extraction
- Comparing text similarity



Decoder-only Transformer

Generate creative content

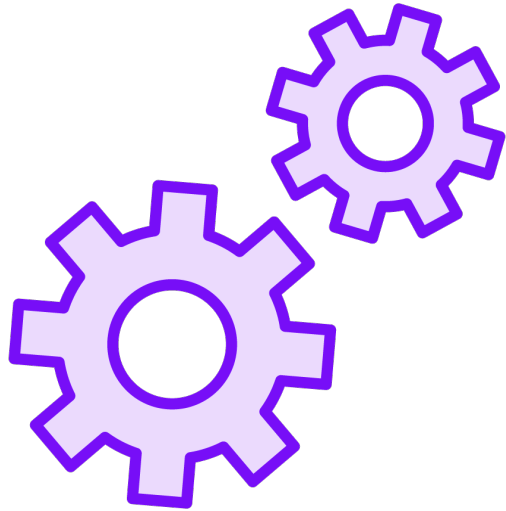
GPT-n models

Autoregressive text generation

Predict next word in sequence



Encoder-decoder Transformer



BART, Text-to-Text Transfer Transformer (T5)

Process input, generate output

Use cases:

- Language translation
- Summarization
- Conversational chatbot





Developer Resources for Transformers





Text Generation Model Parameters





Prompt

Sets the context for transformer model output



System, User, and Assistant Prompts

Relevant for chat models like Claude 3.5 and Gemini 2.5 Pro



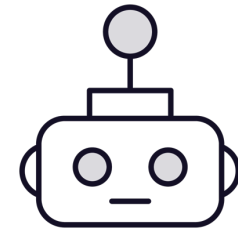
System prompt

Role, personality, tone,
and constraints



User prompt

Instructions or queries
from the user



Assistant prompt

Model's response to
user input



Tuning Transformer Outputs



Max length / max new tokens



Temperature



Do sample



Top-k



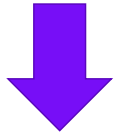
Top-p



Adjusting Temperature Parameter

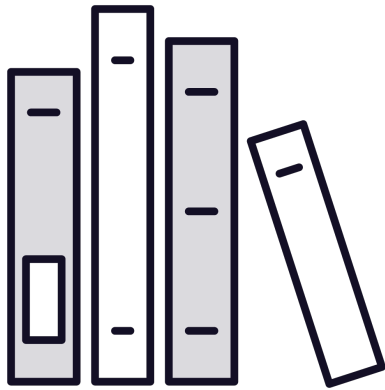


Use high values for creative, varied content



Use low values for consistent, accurate responses





Where Can You Learn More?

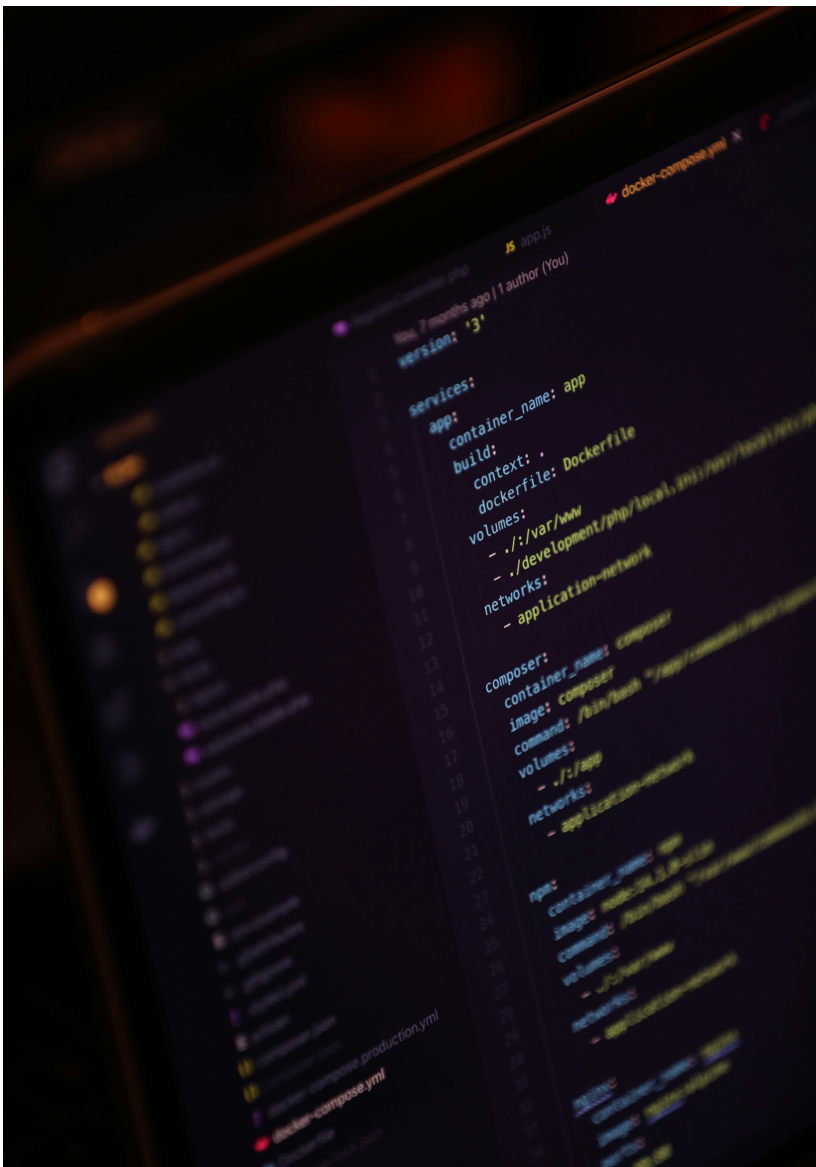
Hugging Face Transformers documentation
<https://huggingface.co/docs/transformers/en/index>





Code Generation Models





Specialized LLMs for producing code

Training data includes massive code bases:

- Github
- Stack Overflow
- Documentation
- Open-source projects

Learn programming syntax, semantics and patterns

RLHF aligns outputs with coding standards

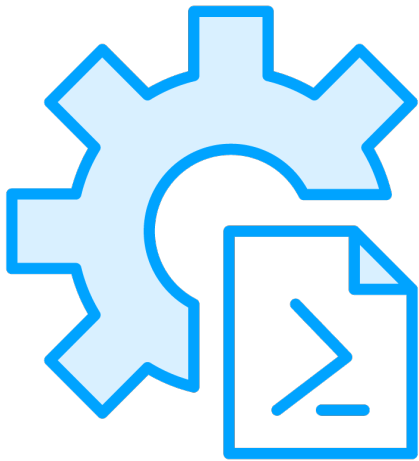


General-purpose Code Models

- Predict next line/block of code
- Automate repetitive tasks
- Create unit/integration tests
- Suggest error fixes & best practices
- Uses natural language prompts
- Not deeply specialized for coding tasks



Specialized Code Models



Trained specifically on large-scale codebases

Specialized techniques and architectures:

- Token-level attention mechanisms
- Abstract syntax tree processing
- Specialized code embeddings

Examples:

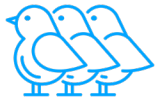
- Code Llama
- StarCoder
- CodeT5



Specialized Coding Tasks



Code search
CodeBERT and GraphCodeBERT



Clone detection
GraphCodeBERT



Code defect and vulnerability detection
VulBERTa and CodeReviewer



Code translation and summarization
Transcoder and CodeT5



Using General-purpose vs. Specialized Models

General-purpose models

Quick prototyping and exploratory programming

Conversational coding assistance

Drafting technical documentation

vs.

Specialized models

High precision and reliability in code generation

Structural or syntactic code understanding

Code search, clone detection, defect detection





Image Generation Models





What Are Diffusion Models?

Process:

- Format images for training
- Forward diffusion: gradually add noise
- Reverse diffusion: iteratively remove noise

Generates new images from noise

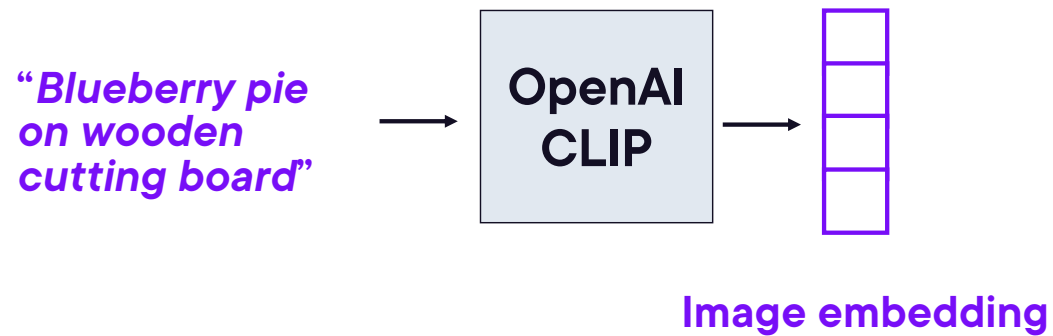


Text-to-image generation
using diffusion

DALL-E

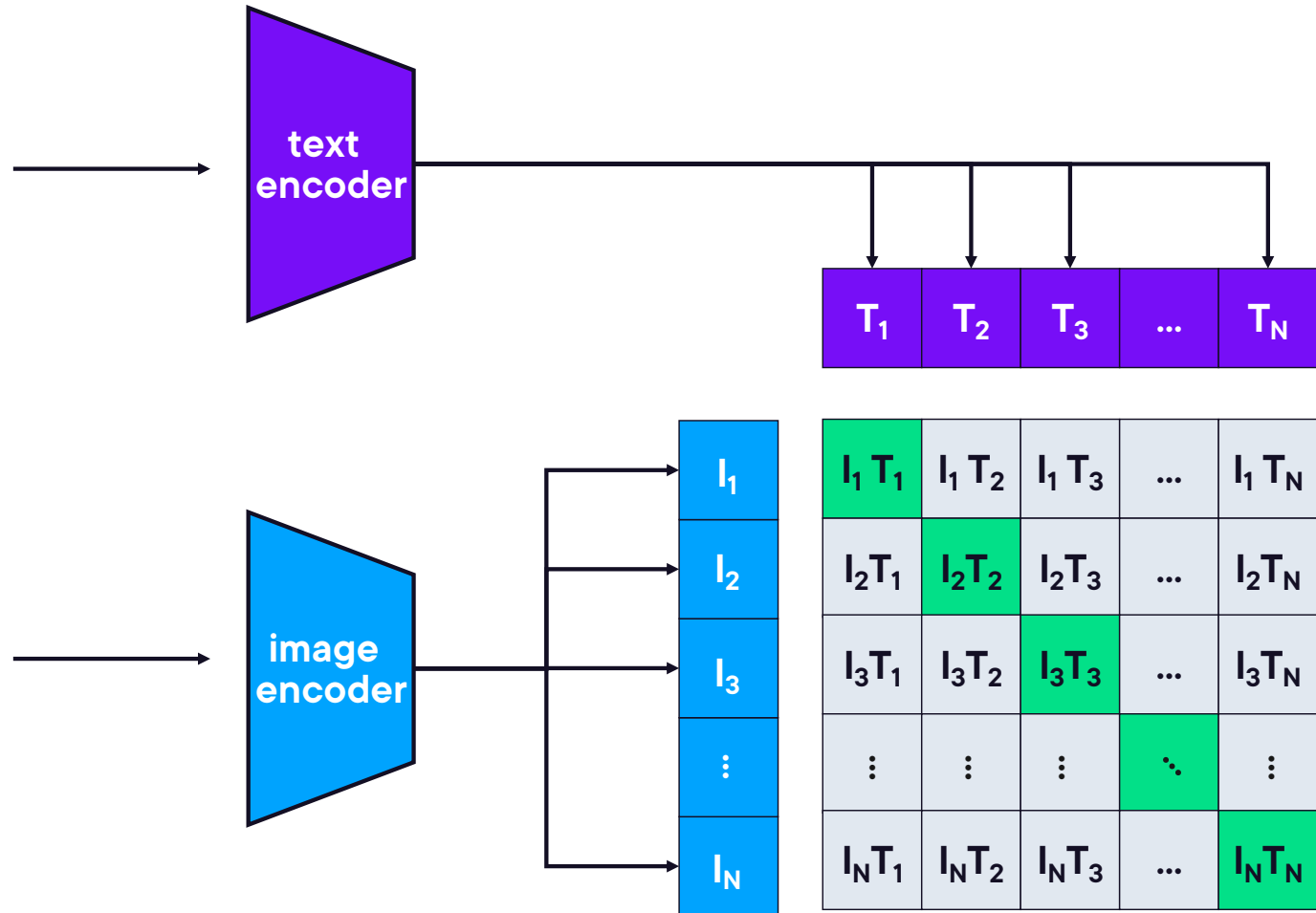


Encoding Text to Image Representations

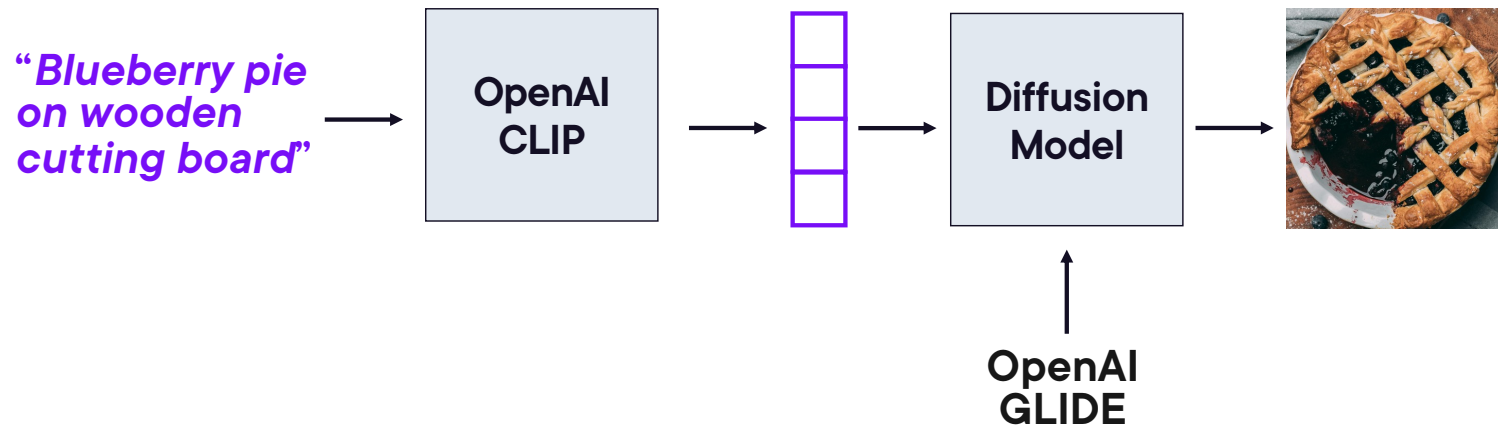


OpenAI CLIP Architecture

*“Blueberry pie
on wooden
cutting board”*



Convert CLIP Embeddings to Images



**Open-source image
diffusion model**

Variational autoencoder

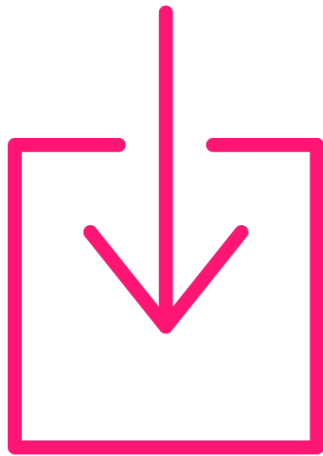
U-Net architecture

CLIP model

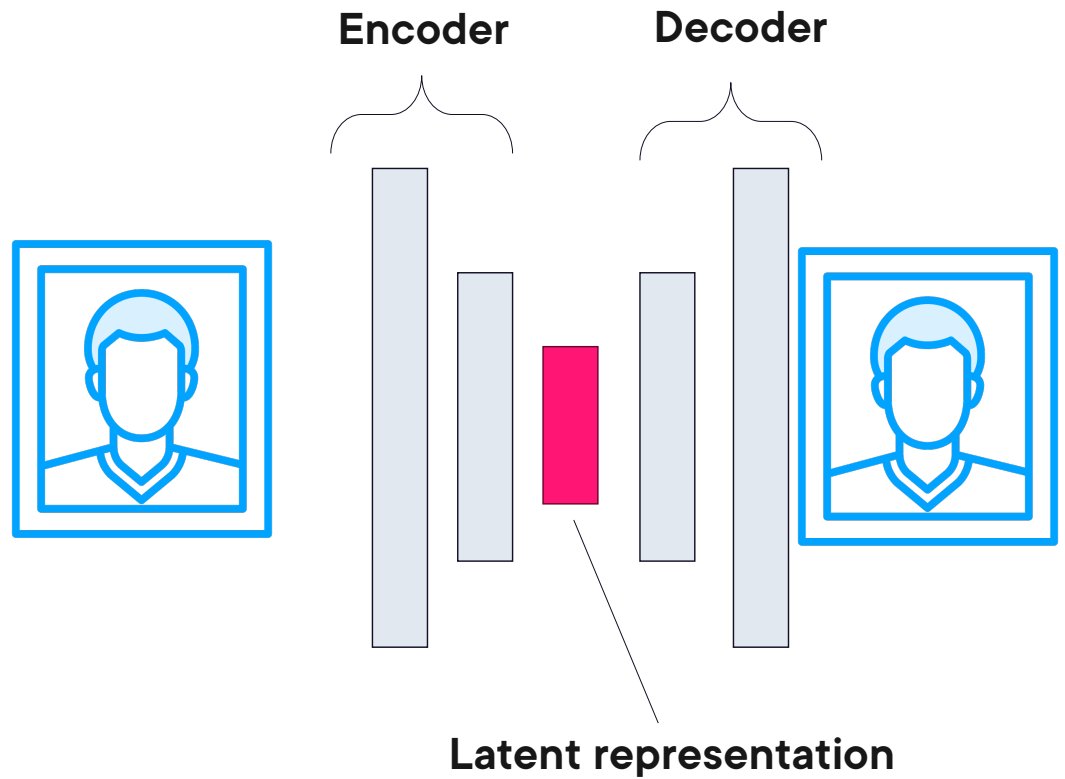
Stable Diffusion



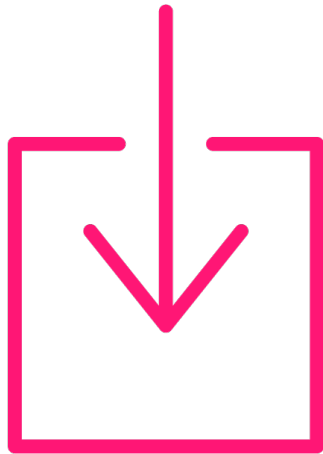
Stable Diffusion Training Process



Train VAE to distill images
into a latent representation



Stable Diffusion Training Process



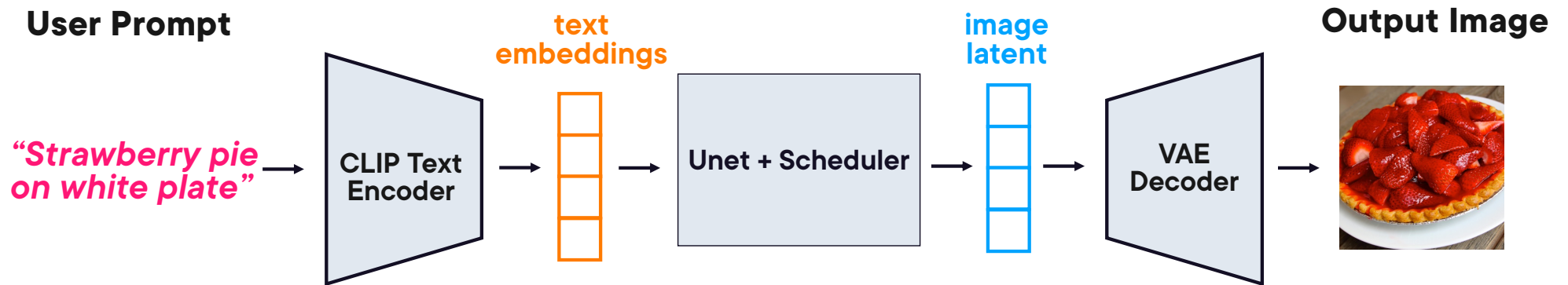
**Train VAE to distill images
into a latent representation**



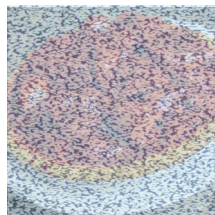
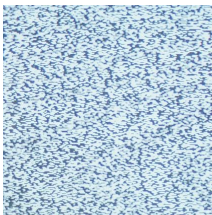
**U-Net learns to denoise
CLIP embeddings**



Stable Diffusion Architecture



Diffusion Process



...



Stable Diffusion Parameters



Negative prompt



Width & height



Steps



Scheduler

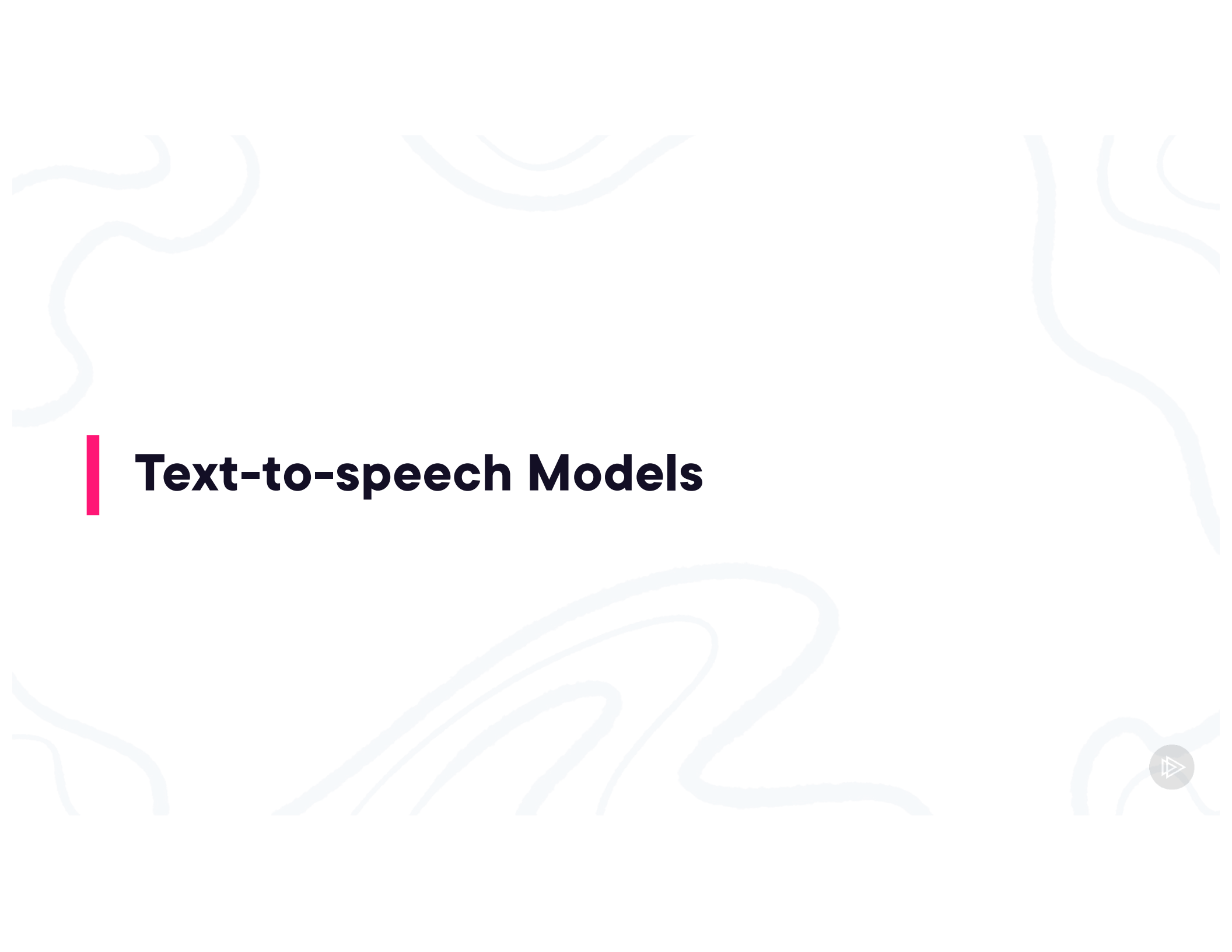


CFG Scale



Seed

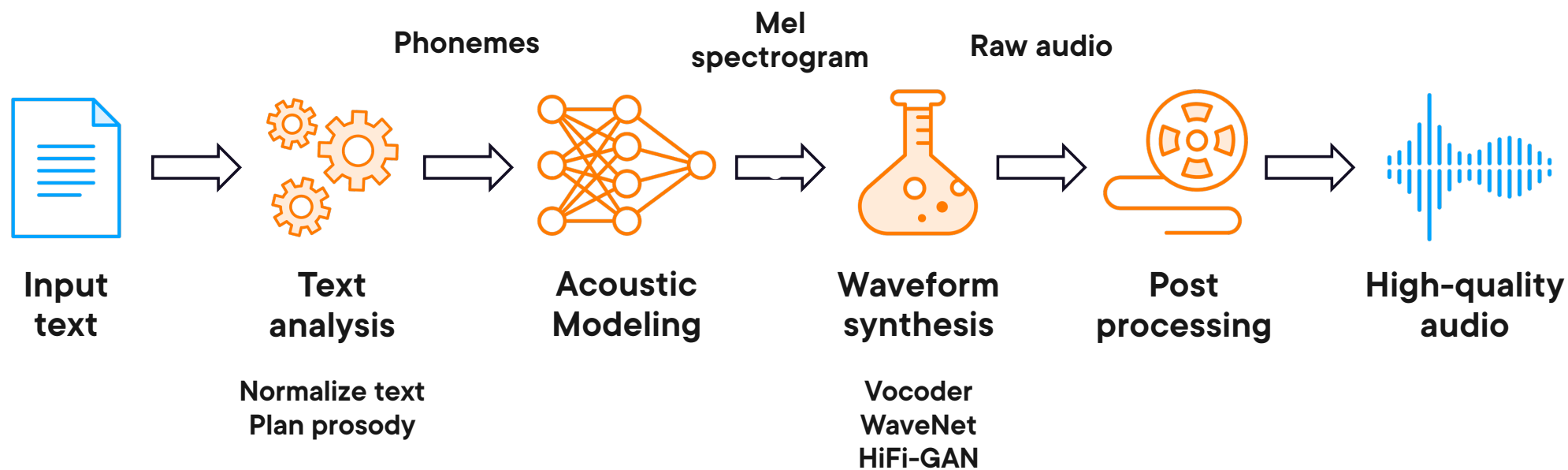


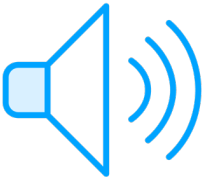
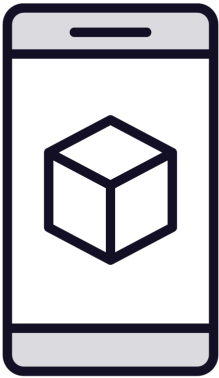


Text-to-speech Models



How Text-to-speech Models Work







Multimodal Large Language Models



Multimodal AI

Machine learning models capable of processing and integrating information from multiple modalities or types of data.

<https://www.ibm.com/think/topics/multimodal-ai>



Multimodal Model Examples



DALL-E



Chatterbox



Multimodal Large Language Model (MLLM)

LLM that can process different types of data

Fuse and interpret info from multiple sources

Tasks

- Image captioning
- Visual question and answering
- Read and extract data from PDFs
- Execute spoken instructions



Multimodal LLM Architectures

**Unified embedding decoder
architecture**

**Cross-modality attention
architecture**

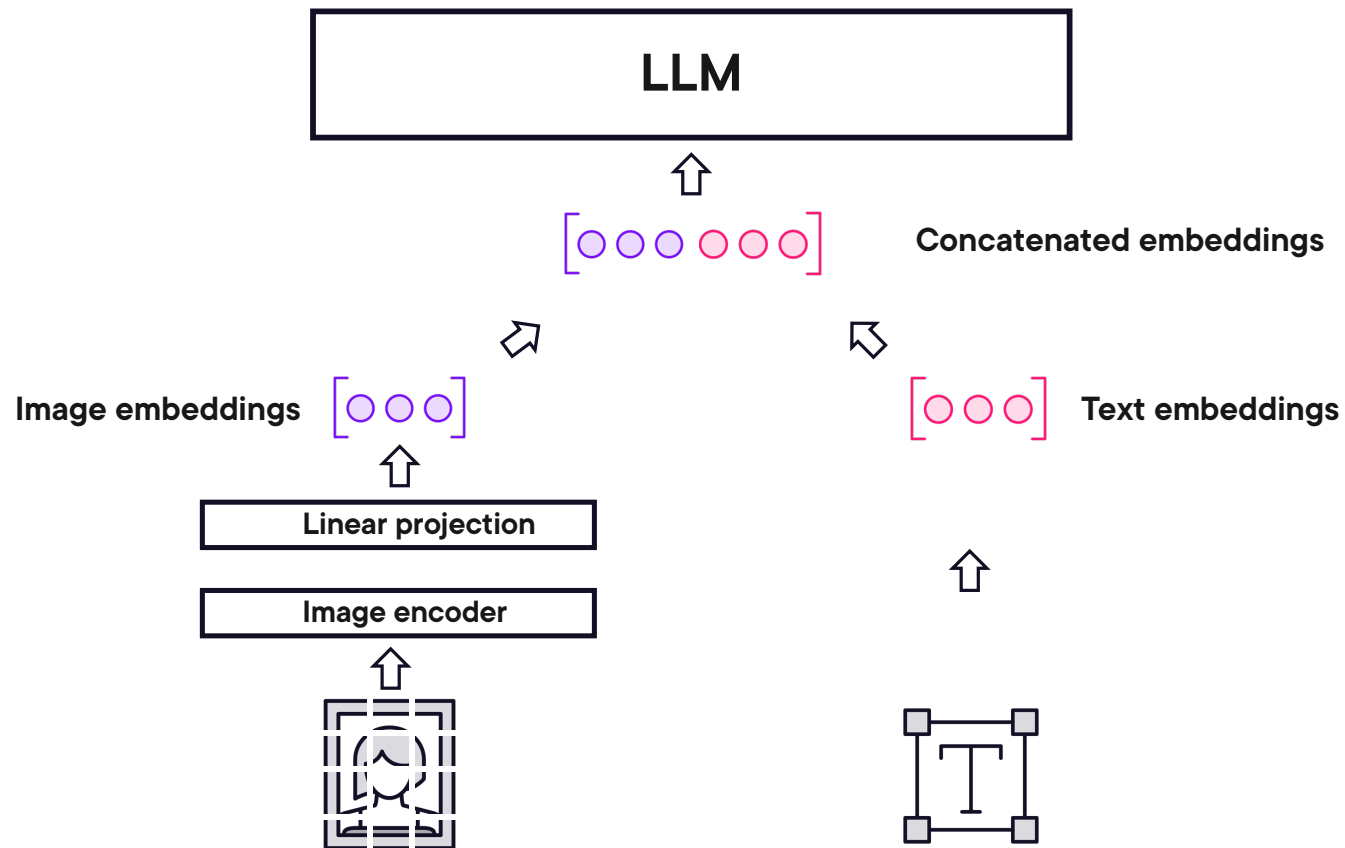


Unified Embedding Decoder Architecture

Example MLLMs: Pixtral 12b and Fuyu



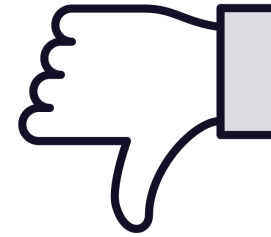
Unified Embedding Decoder Architecture



Unified Embedding Decoder Architecture



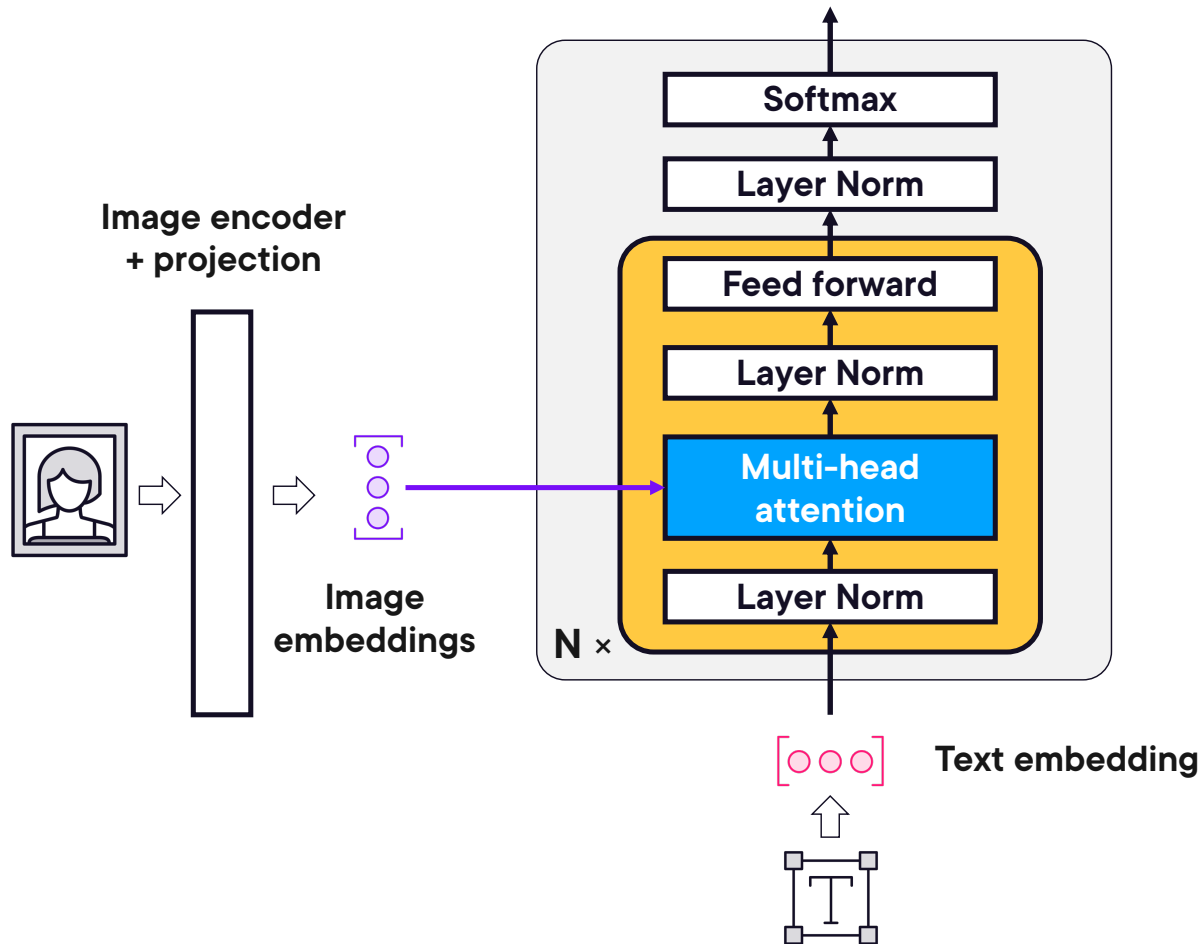
Simplicity
No changes to LLM needed



Fills context window quickly
Loses some visual information
Can't focus on specific regions



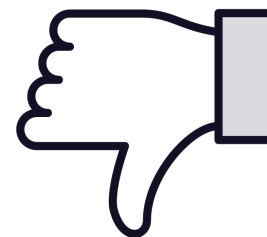
Cross-modality Attention Architecture



Cross-modality Attention Architecture



No overloading of context window
Selectively attend to image



Requires changes to architecture
More training effort

Example MLLMs: Flamingo, MiniGPT-4, and LLaVA



Training Multimodal LLMs

1 Alignment

Start with pretrained LLM and modality encoders (e.g., CLIP, ViT)
Train adapters to align modalities to LLM internal representation

2 Fine-tuning

Unfreeze some MLLM components
Fine-tune system on downstream multimodal tasks







Parameter-efficient Fine-tuning





7B model

Full Fine-tuning Large Models

Hundreds of GBs of GPU memory

Days-weeks of compute

Massive task-specific checkpoints

Risk of catastrophic forgetting



What Is Parameter-efficient Fine-tuning?

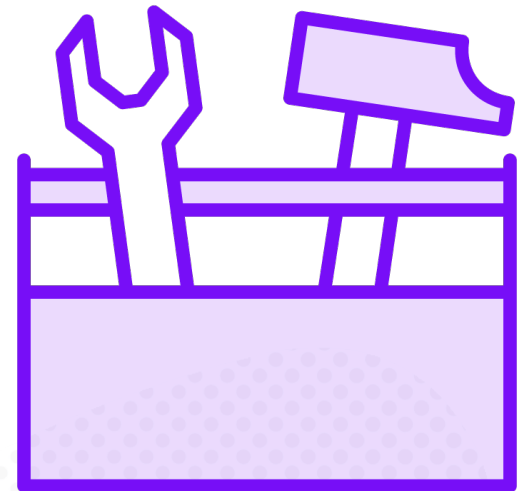
Attach lightweight modules to LLM

10x to 100x lower memory usage

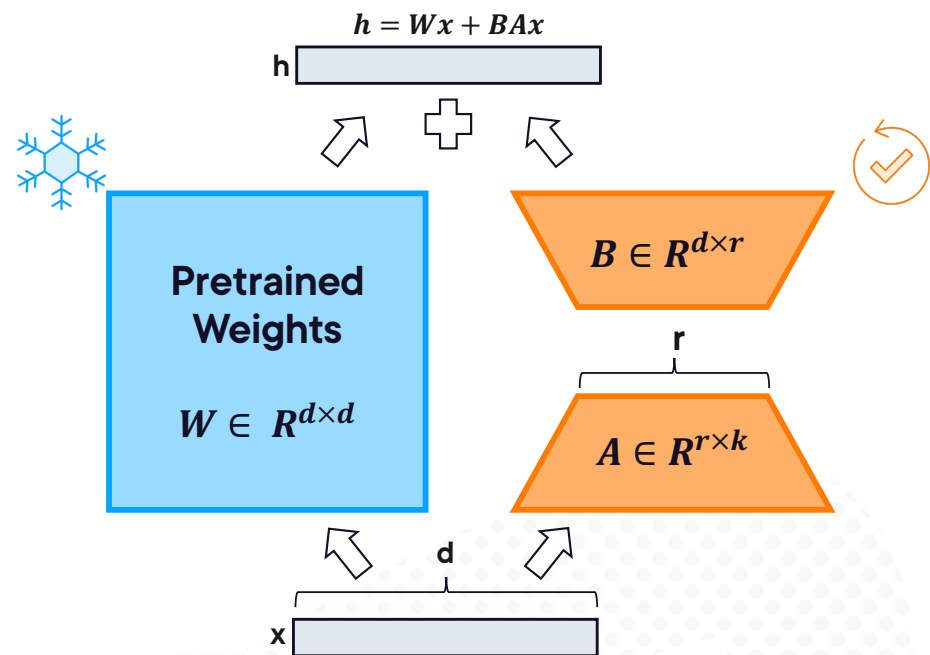
Tiny storage footprint

Preserved general knowledge

Can match or outperform full fine-tuning



Low-rank Adaptation (LoRA)



Low-rank Adaptation (LoRA)

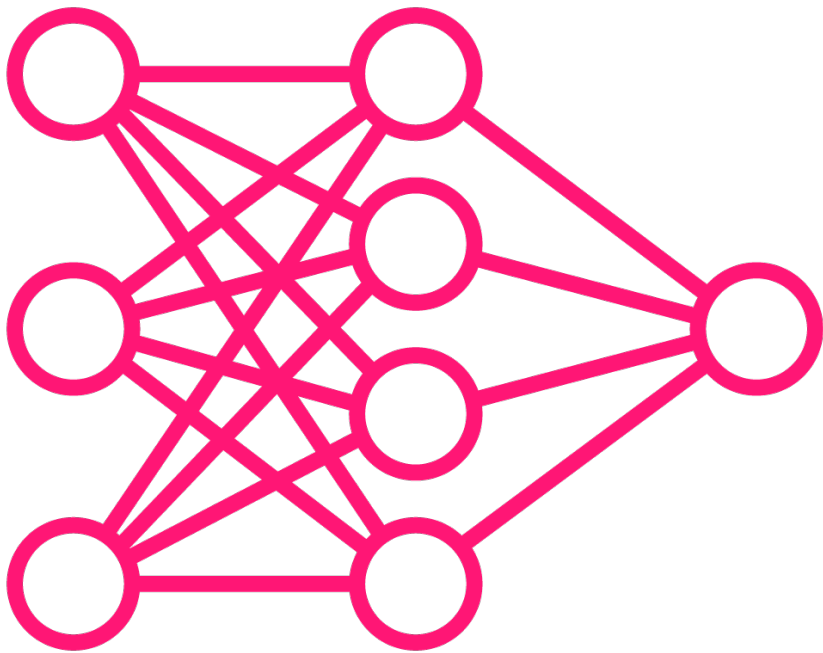
Rank selection matters

- Too low, model underfits
- Too high, training becomes inefficient

Minor accuracy loss, but negligent



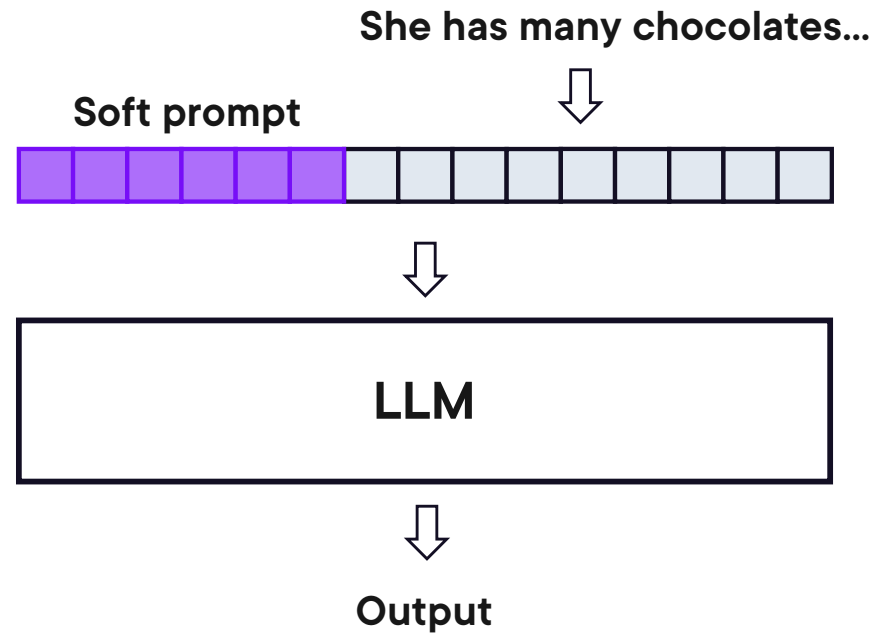
Quantized Low-rank Adaptation (QLoRA)



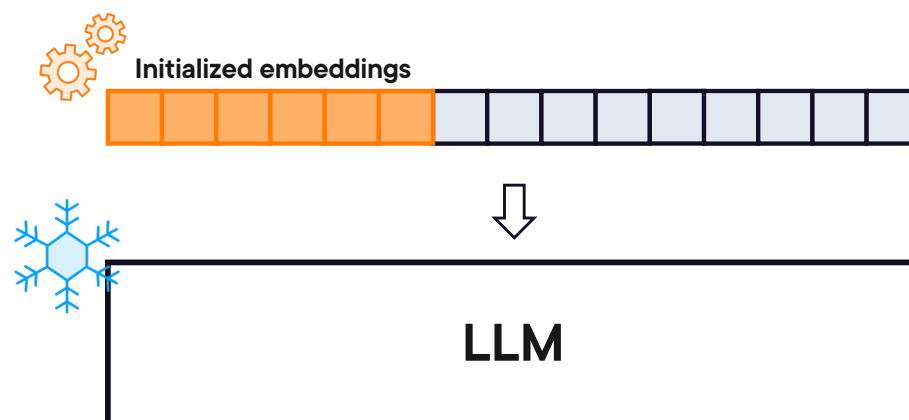
Fine-tune 65B+ models on 48GB GPU
Uses less memory than standard LoRA
Maintains accuracy



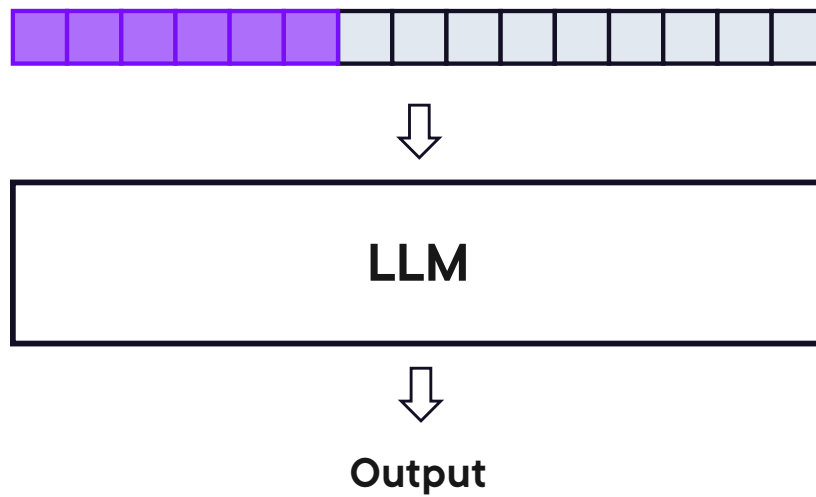
Prompt Tuning



Prompt Tuning



Prompt Tuning





Friendly soft prompt



“Hey there! So glad you dropped by—what can I do for you?”





Professional soft prompt



***“Thank you for contacting Bethany’s Pie Shop.
How may I assist you?”***



Prompt tuning is great for

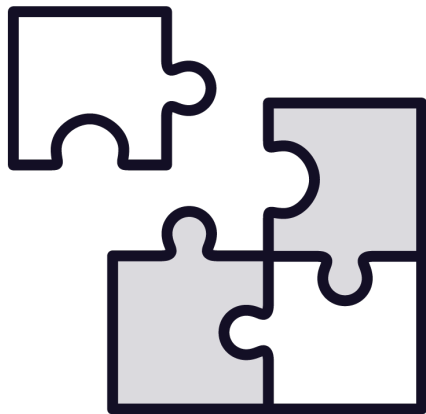
Rapid task prototyping

Text classification

Style or tone adaptation

Working with frozen, large-scale models



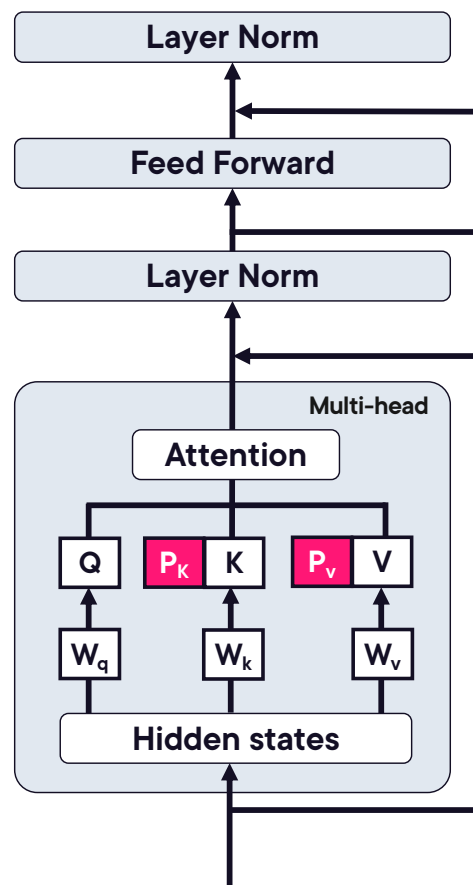


Prefix Tuning

Injects trainable vectors into attention mechanisms of each layer



How Prefix Tuning Works





Summarizing Online Reviews

Select detailed flavor notes

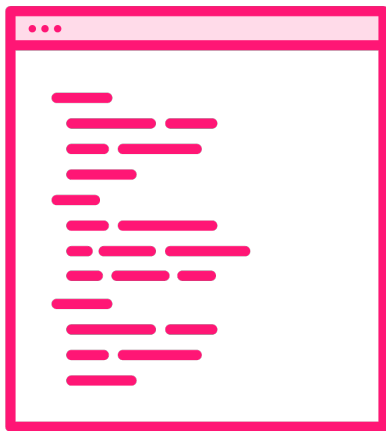
Recognize customer sentiment

Identify complaints, compliments and suggestions

Prefix tuning is ideal for:

- Technical summarization
- Domain-specific generation
- Structured reasoning tasks





Hugging Face PEFT Library

Implements parameter-efficient fine-tuning methods

Documentation:

<https://huggingface.co/docs/peft/index>





Generative Model Cost Estimation

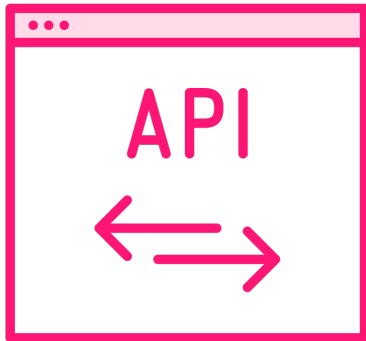


Visualize Your Dream Pie Service



“Cherry pie with golden lattice crust”





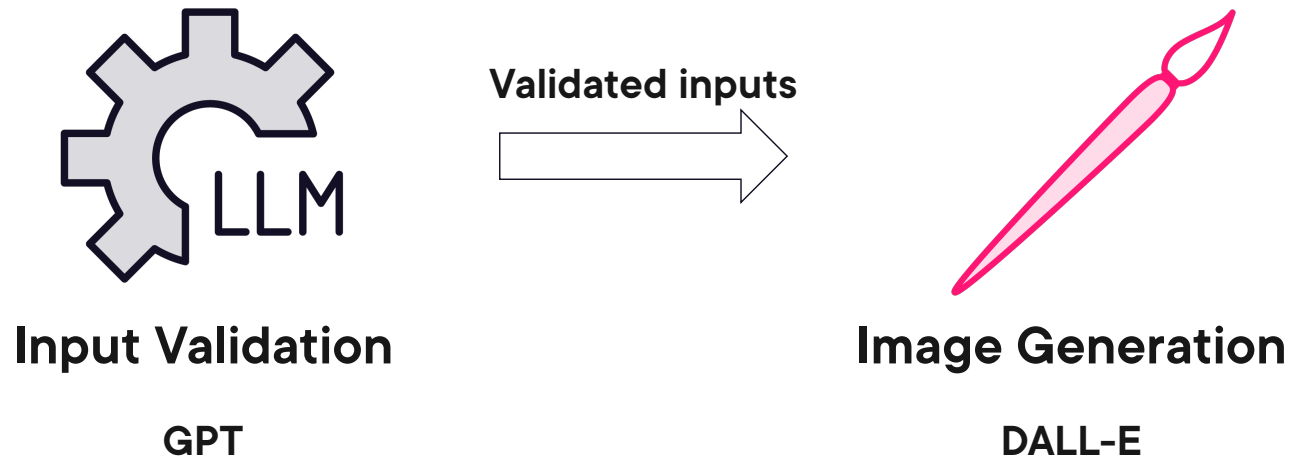
Hosted APIs

OpenAI, Anthropic, Google, etc.

No infrastructure setup



Dream Pie Visualization with OpenAI



```
pip install tiktoken
```

Tiktoken Library

Fast open-source library for converting text into tokens



Dream Pie Visualization with OpenAI

Monthly estimate: 1000 custom pie visualizations

Image generation cost: \$0.02 per image

GPT screening cost: \$0.000211 per request

Total monthly cost: \$20.21





Self-hosting

Llama, Mistral, Stable Diffusion, etc.

More control and flexibility over implementation

Hosting cost = # hosting hours × hosting rate per hour



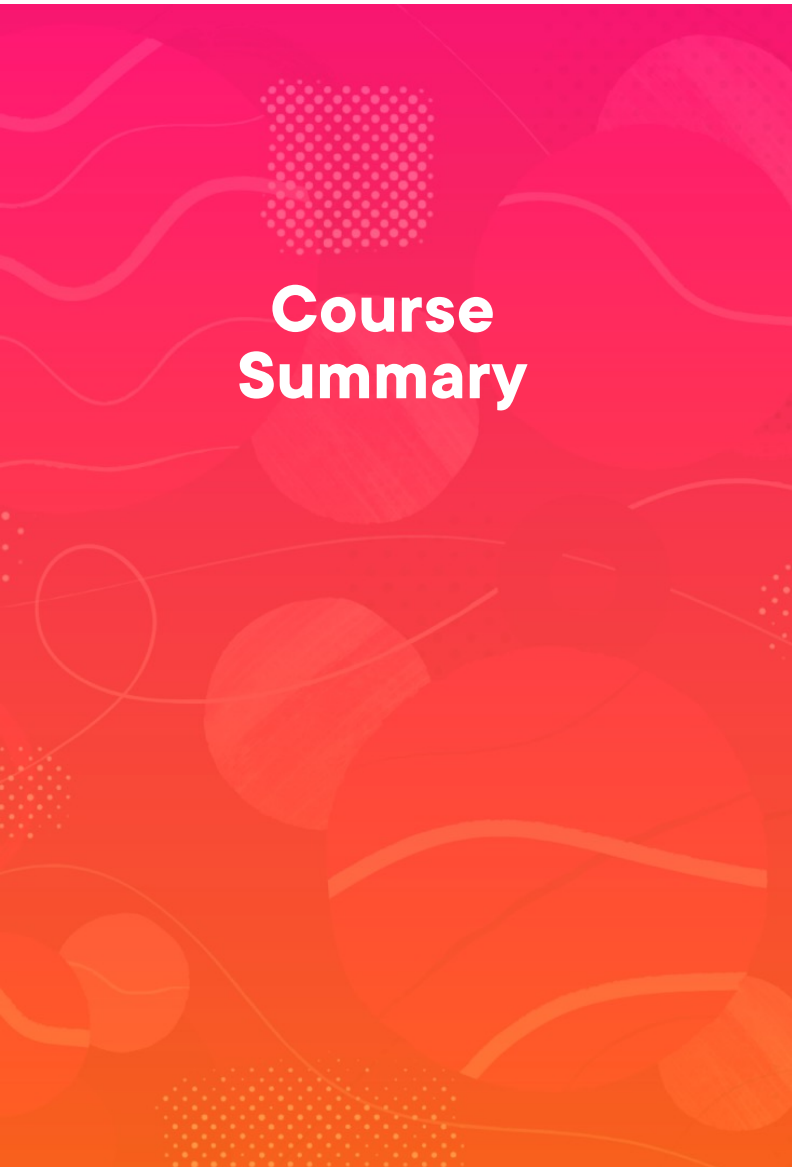
Self-hosted Dream Pie Visualization

NVIDIA A10 GPU server cost: \$0.75 per hour

Monthly hosting cost: \$504

Fine-tuning cost: \$7.50 (assuming 10 hr. training time on A10 GPU) per image





Course Summary

How generative AI works

Tuning model performance

Estimating project costs

